Inversion of Control Containers and the Dependency Injection

What are the different types of IOC (dependency injection)?

There are three types of dependency injection:

- **Constructor Injection** (e.g. Pico container, Spring etc): Dependencies are provided as constructor parameters.
- **Setter Injection** (e.g. Spring): Dependencies are assigned through JavaBeans properties (ex: setter methods).
- **Interface Injection** (e.g. Avalon, ServiceLocator): Injection is done through an interface.

  Note: Spring supports only Constructor and Setter Injection

Constructor Injection with PicoContainer

Setter Injection with Spring

Interface Injection Using Service Locator

Method Injection in Spring

For most users, the majority of the beans in the container will be singletons. When a singleton bean needs to collaborate with (use) another singleton bean, or a non-singleton bean needs to collaborate with another non-singleton bean, the typical and common approach of handling this dependency by defining one bean to be a property of the other, is quite adequate. **There is however a problem when the bean lifecycles are different**. Consider a singleton bean A which needs to use a non-singleton (prototype) bean B, perhaps on each method invocation on A. The container will only create the singleton bean A once, and thus only get the opportunity to set its properties once. There is no opportunity for the container to provide bean A with a new instance of bean B every time one is needed.

One solution to this problem is to forgo some inversion of control. Bean A can be aware of the container by implementing `BeanFactoryAware`, and use programmatic means to

ask the container via a `getBean("B")` call for (a new) bean B every time it needs it. This is generally not a desirable solution since the bean code is then aware of and coupled to Spring.

Method Injection, an advanced feature of the BeanFactory, allows this use case to be handled in a clean fashion, along with some other scenarios.

1. Lookup method Injection

Lookup method injection refers to the ability of the container to override abstract or concrete methods on managed beans in the container, to return the result of looking up another named bean in the container. The lookup will typically be of a non-singleton bean as per the scenario described above (although it can also be a singleton). Spring implements this through a dynamically generated subclass overriding the method, using bytecode generation via the CGLIB library.

In the client class containing the method to be injected, the method definition must be an abstract (or concrete) definition in this form:

```
protected abstract SingleShotHelper createSingleShotHelper();
```

If the method is not abstract, Spring will simply override the existing implementation. In the XmlBeanFactory case, you instruct Spring to inject/override this method to return a particular bean from the container, by using the `lookup-method` element inside the bean definition. For example:

```
<!-- a stateful bean deployed as a prototype (non-singleton) -->
<bean id="singleShotHelper class="..." singleton="false">
</bean>

<!-- myBean uses singleShotHelper -->
<bean id="myBean" class="...">
  <lookup-method name="createSingleShotHelper" bean="singleShotHelper"/>
  <property>
    ...
  </property>
</bean>
```

The bean identified as *myBean* will call its own method `createSingleShotHelper` whenever it needs a new instance of the *singleShotHelper* bean. It is important to note that the person deploying the beans must be careful to deploy *singleShotHelper* as a non-singleton (if that is actually what is needed). If it is deployed as a singleton (either explicitly, or relying on the default *true* setting for this flag), the same instance of singleShotHelper will be returned each time!

Note that lookup method injection can be combined with Constructor Injection (supplying optional constructor arguments to the bean being constructed), and also with Setter Injection (settings properties on the bean being constructed).

2. Arbitrary method replacement

A less commonly useful form of method injection than Lookup Method Injection is the ability to replace arbitrary methods in a managed bean with another method implementation. Users may safely skip the rest of this section (which describes this somewhat advanced feature), until this functionality is actually needed.

In an XmlBeanFactory, the `replaced-method` element may be used to replace an existing method implementation with another, for a deployed bean. Consider the following class, with a method computeValue, which we want to override:

```
...
public class MyValueCalculator {
  public String computeValue(String input) {
    ... some real code
  }

  ... some other methods
}
```

A class implementing the `org.springframework.beans.factory.support.MethodReplacer` interface is needed to provide the new method definition.

```
/** meant to be used to override the existing computeValue
    implementation in MyValueCalculator */
public class ReplacementComputeValue implements MethodReplacer {

    public Object reimplement(Object o, Method m, Object[] args) throws
Throwable {
        // get the input value, work with it, and return a computed
result
        String input = (String) args[0];
        ...
        return ...;
}
```

The BeanFactory deployment definition to deploy the original class and specify the method override would look like:

```
<bean id="myValueCalculator class="x.y.z.MyValueCalculator">
  <!-- arbitrary method replacement -->
  <replaced-method name="computeValue"
replacer="replacementComputeValue">
    <arg-type>String</arg-type>
  </replaced-method>
</bean>

<bean id="replacementComputeValue"
class="a.b.c.ReplaceMentComputeValue"/>
```

One or more contained `arg-type` elements within the `replaced-method` element may be used to indicate the method signature of the method being overridden. Note that the signature for the arguments is actually only needed in the case that the method is actually overloaded and there are multiple variants within the class. For convenience, the type string for an argument may be a substring of the fully qualified type name. For example, all the following would match *java.lang.String*.

```
java.lang.String
String
Str
```

Since the number of arguments is often enough to distinguish between each possible choice, this shortcut can save a lot of typing, by just using the shortest string which will match an argument.

Annotations for Dependency Injection

There are 4 annotations for dependency injection in Spring Framework:

- `@Autowired`,
- `@Configurable`,
- `@Qualifier`,
- `@Resource`.

The `@Autowired` annotation autowires **by type**, but it can also be used along with the `@Qualifier` annotation to provide more specific autowiring behavior if there will be multiple beans of the same type in the IoC container.

The `@Configurable` annotation is used to mark a class eligible for Spring dependency injection, but it's typically used with classes instantiated outside of the IoC container. This annotation is used along with the context:spring-configured element and will be covered in the AOP chapter since it's AOP related.

The `@Resource` annotation autowires **by name** and is from JSR-250, which is the specification for Commons Annotations.

Table 4.3. Annotations from `org.springframework.beans.factory.annotation`

Annotation	Since Version	Target	Description
`@Autowired`	Spring 2.5	Constructor, Field, or Method	If the `AutowiredAnnotationBeanPostProcessor` is registered with the IoC conatiner as the bean is processed, the constructor, field, or method is autowired **by type**.
`@Configurable`	Spring 2.0	Class	Indicates a class is eligible for Spring configuration, but is typically only used with AspectJ and along with *context:spring-configured*.
`@Qualifier`	Spring 2.5	Field or Parameter	

Table 4.4. Annotations from `javax.annotation`

Annotation	Since Version	Target	Description
`@Resource`	Spring 2.5	Field or Method	If the `CommonAnnotationBeanPostProcessor` is installed as the bean is processed, the field or method is autowired **by name**. Part of JSR-250 (Commons Annotations).

@Autowired with Constructors, Fields, and Methods

Without using any other annotations, **`@Autowired`** can autowire a constructor, field, or method **by type**. The `AutowiredAnnotationBeanPostProcessor` must be registered with either the `BeanFactory` or `ApplicationContext` to have annotation-based autowiring on beans work. This bean post processor is implicitly registered by *context:component-scan* and *context:annotation-config*. Both of which will be covered in more detail in the following section.

When `@Autowired` is unable to find at least one match, a `BeanCreationException` will be thrown. By default it required that it autowires at least one value, but this can be changed by setting `@Autowired(required=false)`. If more than one match is found when matching by type and it isn't an array or `Collection` class, a `BeanCreationException` will be thrown. If there is one bean that should be selected out of many of the same type, the bean element's primary attribute can be set to true. This will let autowiring by type pick a unique bean from a list of beans that are all the same type. There must be only one bean marked as the primary bean for a type or an exception will still be thrown.

One or more parameters can be autowired for a constructor or method. Only one constructor can be marked as required, although more than one can be set as autowired. If there is more than one constructor marked as autowired, Spring will use the constructor

that can have the most arguments matched based on the beans in the IoC container and the autowiring rules.

Example 4.1. @Autowired Constructor

The two different parameters will be autowired by type. As previously mentioned, there must only be one bean defined in the container of each type being autowired or there will be an error.

Excerpt from chapter04-annotation-based-autowire/src/main/java/org/springbyexample/springindepth/chapter04/annotationBasedAutowire/ConstructorContainer.java

```
@Autowired
public ConstructorContainer(MockDatasource mockDatasource,
MockRemoteService mockRemoteService) {
```

Example 4.2. @Autowired Field

Based on the type of the field, a MockDatasource bean is set on the field. There must be only one matching bean.

Excerpt from chapter04-annotation-based-autowire/src/main/java/org/springbyexample/springindepth/chapter04/annotationBasedAutowire/AutowiredTest.java

```
@Autowired
protected MockDatasource mockDatasource = null;
```

Example 4.3. @Autowired Method

The autowired method has a MockDatasource bean set on the method. There must be only one matching bean.

Excerpt from chapter04-annotation-based-autowire/src/main/java/org/springbyexample/springindepth/chapter04/annotationBasedAutowire/AutowiredTest.java

```
protected MockDatasource methodMockDatasource = null;

@Autowired
public void setMockDatasource(MockDatasource methodMockDatasource) {
    this.methodMockDatasource = methodMockDatasource;
}
```